Provider Network Access and Cost Planner, Part 3: Neural and Probabilistic Models for High-Cost Prediction and Document Reading

Andy Romero, Sharon Arias, Diego De La Vega, David Satizabal

Miami Dade College

CAI4510C: Machine Intelligence

Project Part 3: Neural Models Integration

Dr. Martins Idahosa

November 9, 2026

Provider Network Access and Cost Planner, Part 3: Neural and Probabilistic Models for High-Cost Prediction and Document Reading

In Part 2, we put together a pipeline to flag health plan members who might run up high costs the following quarter. Balanced logistic regression came out on top. Those flags go into our AI planner, which matches members to doctors while trying not to overload any one provider with the toughest cases. For Part 3, we wanted to see if a neural network could actually beat that simpler model. We also tried a CNN on the scanned forms from our earlier AI project, and a probabilistic approach to making sense of stretches of high spending. Everything here is synthetic, generated with seed 42. No actual member records, providers or claims went into it.

Problem and Data Snapshot

We stuck with our term project data rather than switching over to the small classroom datasets. That way whatever worked could go back into the same planner. The claims data started with 31,563 member month records covering 1,500 members, from January 2025 through September 2026. We found and removed 63 duplicates. We marked a row high cost if spending in the next three months reached $3,904.47 or more, which was the top 10% cutoff in training. We split by date instead of randomly: January through September 2025 for training (13,500 rows), a three month gap, then January and February 2026 for testing (3,000 rows). Of those test rows, 305 were high cost, or 10.2%.

For early stopping, we held back 20% of the training members rather than grabbing random rows. This matters because we didn't want different months from the same person showing up in both groups. That gave me 10,800 rows to fit the model, including 1,035 high cost rows, and another 2,700 for validation, including 315 high cost. After preprocessing, each row had 17 features. Nine were scaled numeric fields, covering things like age, health conditions,

distance to a primary care doctor, visits, ER use, admissions and three spending totals. Sex, county and plan type were one hot encoded, with missing values marked Unknown. We fit those transformations only on the fitting group.

For images, we had 1,500 fake scans at 64 by 64 pixels, all grayscale. The three classes were claim forms, referrals and enrollment forms. We used 840 images for training, 180 for validation and 180 for the regular test. We also made 300 deliberately messy scans with things like blur, tilt, noise and uneven lighting. None of those messy images were used to train either model.

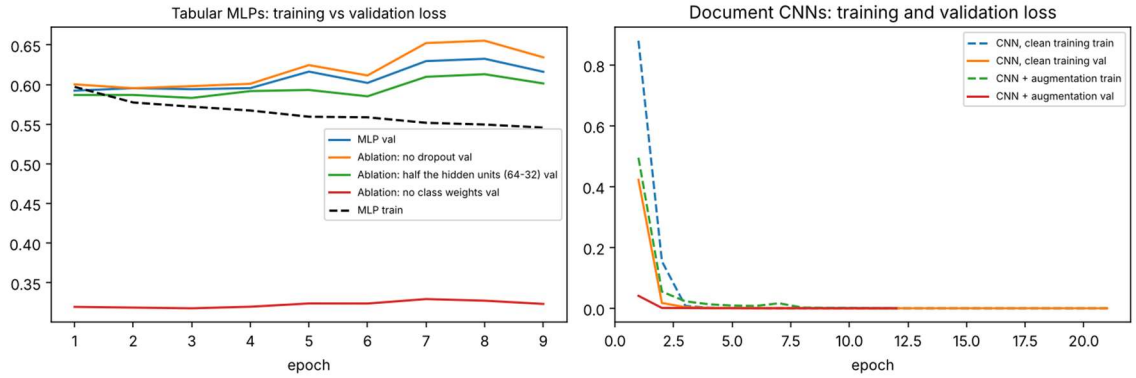
Architectures and Hyperparameters

Multilayer Perceptron

We kept the MLP pretty close to the assignment. It takes 17 inputs, runs them through 128 neurons with ReLU and 20% dropout, then 64 more with ReLU, and finally produces two outputs. Total parameters: 10,690.we used Adam with a 0.001 learning rate and 0.0001 weight decay, batches of 64, and a ceiling of 50 epochs. Early stopping watched validation ROC AUC with a patience of 8.we also weighted the cross entropy loss by class, since only about one in ten rows is high cost. Part 2 already showed what happens without weights: the model mostly says no. Validation ROC AUC hit its best value, 0.760, after just one epoch. Training ended after epoch 9 and we kept the first epoch's weights. Figure 1 makes the reason pretty obvious. Training loss kept slipping from 0.597 to 0.546 but validation loss went up. More training was not helping, it was learning the training members too closely.

Figure 1

Training and Validation Loss for the Tabular MLPs and the Document CNNs



Note. On the left, validation loss for the MLP starts rising after epoch 1 even though training loss drops. On the right, the two CNNs get close to zero loss within a few epochs. The complete logs are in outputs/training_logs.csv.

Convolutional Neural Network

The CNN uses two convolution blocks with 3 by 3 filters, first 8 then 16, each followed by ReLU and 2 by 2 max pooling. After that comes a dense layer with 32 units and three outputs. It has 132,451 parameters, mostly sitting in that first dense layer. We scaled the images so ink reads as 1 and blank paper as 0. Training used Adam at 0.001, batches of 32 and up to 50 epochs, with early stopping after 6 epochs without better validation loss. We ran it twice. One version saw only the 840 clean training scans. The other saw 2,520 images, because we added two altered copies per original using tilt, shifts, blur, noise and lighting changes.

Architecture Rationale

We started with an MLP because it was a pretty direct comparison with Part 2. Same inputs, same split and same class weighting. So if it did better there would be a decent argument that the network learned interactions the straight line model missed.

We chose the CNN instead of the LSTM mainly because of the documents. A claim form looks like a claim form partly because of where the boxes, headings and signature lines sit on the page. CNNs work with nearby pixels, picking up small patterns and finding them in other spots

too. Pooling also helps when something shifts a little. The softmax model we used in AI Part 2 treats every pixel like a separate input. Move a checkbox three pixels and suddenly it's looking at a different pattern. LSTMs make more sense for sequences, like month by month bills. But our member features already summarize recent spending, so we used an HMM for the sequence side instead.

That CNN advantage showed up most clearly when the training images were clean. On the messy scans, the clean trained CNN got 90.3% right, while the softmax managed 78.0% (Table 2). Once we gave the softmax shifted, blurred and noisy training copies it jumped to 99.7%. So the CNN's assumptions about images helped when the examples were limited, but adding variety to the training data helped the simpler model even more. For the 17 column claims table, there wasn't a similar advantage. Those features aren't laid out in a meaningful spatial or time order (LeCun et al., 2015).

Metrics and Calibration

Table 1 puts the tabular models side by side on the test months. We refit the Part 2 logistic regression winner on the same 80% training member group used for the MLP, rather than giving either one an easier comparison. It gets a 0.755 test ROC AUC here, compared with 0.752 in Part 2.

Table 1

Comparison Against the Project 2 Champion (High-Cost Flag, Test Months)

Model	Features	Val Acc	F1 (macro)	ROC-AUC	Recall	ECE	Params	Train time	Notes
Project 2 best: logistic regression, balanced	Tabular (17)	.713	.565	.755	.672	.019	18	< 0.1 s	Traditional baseline
MLP, class weights	Tabular	.732	.565	.755	.653	.033	10,690	1.3 s	Kept epoch 1
MLP + MC Dropout (20 passes)	Tabular	.733	.564	.755	.636	.036	10,690	1.3 s	Variance = uncertainty
Gaussian Naive Bayes	Tabular	.844	.616	.736	.298	.108	68	< 0.1 s	Interpretable

Model	Features	Val Acc	F1 (macro)	ROC-AUC	Recall	ECE	Params	Train time	Notes
HMM (2 states)	Monthly cost bands	.829	.590	.673	.253	.010*	14	1.5 s	Readable tables

Note. Recall is the portion of the 305 high cost test rows the model found. ECE uses the assignment's confidence of the chosen answer definition. *The HMM usually predicts no, so its small ECE can be misleading. Figure 2 looks at the high cost probability itself.

The MLP didn't really beat the champion. ROC AUC was 0.7553 versus 0.7554, almost exactly the same. Its recall was a bit worse too, 0.653 versus 0.672, even with roughly 600 times as many parameters. In the test confusion matrix (outputs/mlp_confusion_test.png), it caught 199 of the 305 high cost rows, missed 106 and raised 763 false alarms. Looking at the way we generated this data, that makes sense. Older age, more conditions and higher past spending mostly add to the risk, rather than forming complicated patterns. A flexible network doesn't gain much from that. Research has found similar results with ordinary tabular datasets (Grinsztajn et al., 2022). Naive Bayes looks good if you only look at accuracy, but it's saying no more often and catching just 30% of the high cost group. So I'm sticking with logistic regression.

The document numbers tell a different story (Table 2). With only clean training scans, the CNN beats the softmax on messy images by about 12 percentage points. After augmentation, though, the simple softmax takes the lead, 99.7% to 97.0%. Adding the extra image variations also improves CNN calibration, dropping messy scan ECE from 0.067 to 0.023.

Table 2

Document Type Accuracy, Clean Test Scans and Messy Scans

Model	Training images	Clean test	Messy scans	Messy ECE	Params
AI Part 2 softmax, clean training	840	1.000	0.780		12,291
CNN, clean training	840	1.000	0.903	0.067	132,451
AI Part 2 softmax + augmentation	2,520	1.000	0.997		12,291
CNN + augmentation	2,520	1.000	0.970	0.023	132,451

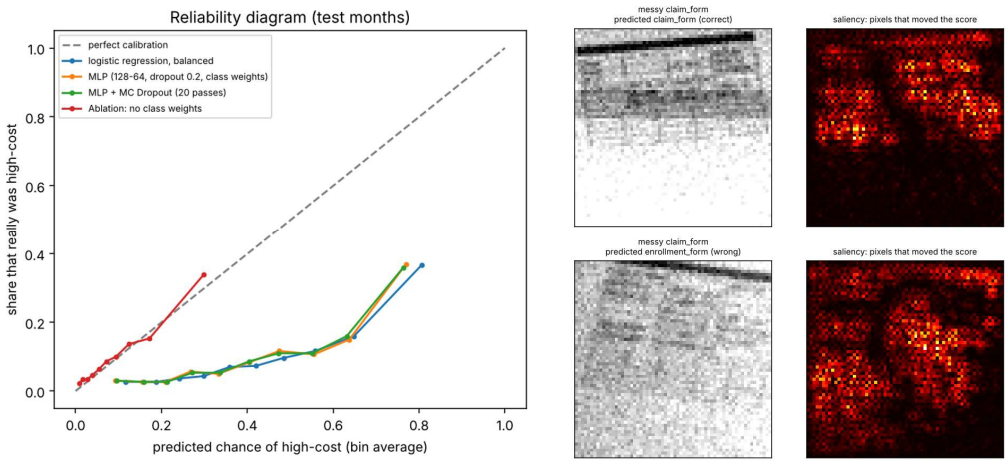
Note. The 300 messy scans were kept completely out of training. The CNN confusion matrix for those scans is saved at outputs/cnn_confusion_messy.png.

Calibration

ECE, as this assignment uses it, checks how confident the model was in whichever class it picked. The balanced logistic regression gets a low 0.019 partly because a lot of its confident no predictions are right. But Figure 2 tells a less comfortable story when we check the actual chance of high cost. When the weighted models predict roughly 60%, only around 15% of those members turn out high cost. At an 80% prediction, the observed rate is closer to 37%. The weights make a missed high cost case count about nine times as much as a false alarm, and that pushes the predicted probabilities up. The unweighted MLP is much closer to the diagonal, but it only catches 3.9% of the high cost rows. Being well calibrated doesn't help much if you miss nearly everybody (Guo et al., 2017).

Figure 2

Reliability Diagram for the Chance of High-Cost (Left) and CNN Saliency Maps for Two Messy Claim Forms (Right)



Note. Left: the points are grouped into quantile bins. Anything below the dashed line overstates the risk. Right: the upper claim form was classified correctly and the lower one incorrectly, using the CNN trained on clean images. Brighter areas had more effect on its score.

Uncertainty Discussion

The calibration results made me change how we use the scores. First, we don't take 0.5 as an automatic yes anymore. At that cutoff, logistic regression would mark 195 of 600 planner

members, which is over three times the actual high cost share. Instead we rank the members and take the highest 10%, or 60 people. Second, the planner uses the numbers to prioritize outreach. It doesn't tell a care manager that someone has a specific percentage chance of being expensive. Third, We set the HMM cutoff the same way, using the riskiest 10% of validation rows, which begins at P = 0.147. Before using any of these as real probabilities I'd need to calibrate them on the validation set, for example with temperature scaling (Guo et al., 2017).

For MC Dropout, we kept dropout switched on during prediction and sent each record through 20 times (Gal & Ghahramani, 2016). The average hardly changed the rankings, ROC AUC was 0.7548, but the differences between runs are useful. They point out members the model can't settle on. Take M1150 in February 2026: age 56, one chronic condition, two ER visits and $60,270 spent in the prior year. Its mean prediction was 0.574 with a standard deviation of 0.167, yet next quarter cost was only $1,841. M1455 was 40, with one condition, one ER visit and $98,114 in past year spending. It averaged 0.337 with a standard deviation of 0.139 and then cost just $143 the following quarter. In both cases a huge bill last year seems to confuse the model because the other warning signs aren't really there. In the integrated planner we added a care team review note for the 60 members with the widest prediction spread. Only 7 of them also land in the top 60 by high cost score. So uncertainty flags a different set of people, and that seems useful.

Visual Explanations

To see what the CNN was paying attention to, we used gradient saliency. Basically it shows how much a pixel affects the score of the class the model chose (Simonyan et al., 2014). The two messy claim forms on the right side of Figure 2 show the difference. For the one it got right, most of the bright spots sit on the boxes near the top, which is the layout clue I'd expect.

For the form it wrongly called an enrollment form, the page is washed out and noisy and the bright spots are scattered all over. This is why augmentation matters. It took messy scan accuracy from 90.3% to 97.0%, and I'd still send uncertain scans to a person instead of guessing.

Probabilistic Model Results

The HMM uses two hidden states for each month: normal and an expensive stretch. we called it a high cost spell when spending over the previous three months landed in the top 10% of the training data, at least $4,034. What the model actually sees is the current month's cost band: below $100, $100 to $999, $1,000 to $4,999, or $5,000 and up. we counted the starting states, transitions and emissions in the training months, then added one to each count so none of the probabilities were zero. In a normal month, the chance of moving into a spell next month is 3.2%. Once somebody is in a spell, the chance of staying there is 68.6%. Also, 22.3% of spell months have bills of $5,000 or more, compared with just 0.01% of normal months. Viterbi gives the most likely sequence of states. For M0558, the monthly bands from January 2025 through February 2026 were 1, 1, 0, 0, 0, 3, 0, 1, 0, 0, 3, 2, 1, 0. The method marks June 2025 and November through December 2025 as spells. A big month followed by quiet months can look like a one off. Two heavy months close together look more like a stretch.

The HMM did worse at predicting high cost than the tabular models, with ROC AUC 0.673 and recall 0.253. It only has bill size bands, so it misses things like age and medical conditions. There's another limitation worth saying plainly: we calculated the supposed hidden state from known costs so we could count the tables. A full HMM would learn those unseen states, for example with Baum Welch (Rabiner, 1989). Gaussian Naive Bayes ranks better at 0.736 and has only 68 parameters, but its ECE is the worst, 0.108. It assumes the inputs are

independent, so the three overlapping spending totals get treated like three separate clues when they're not.

Probabilistic Justification

We can see three situations where Naive Bayes or the HMM might be a better fit. One is when there isn't much data. These models mostly work from counts, and generative models can reach their best error rate with fewer examples than discriminative ones (Ng & Jordan, 2002). My MLP was already past its best point after a single pass through 10,800 rows. Another is when the data changes. In our scenario, Pinecrest Medical Group leaves the network in June 2026.we could recount the HMM tables on newer months quickly, and actually see what shifted, like the chance of entering an expensive spell. With the MLP I'd be looking at changes buried in 10,690 weights. The last reason is explainability. It's easier to tell a care manager that somebody in an expensive stretch has around a 69% chance of staying there, and show where that number came from. Deep learning makes more sense when there's enough data or raw inputs like scanned images.

Ablations and Error Analysis

We took pieces away from the MLP one at a time for Table 3. Losing dropout hardly moved the results. Cutting the hidden layers in half removed about two thirds of the parameters and barely changed anything either. The big difference came from class weights. Without them the accuracy climbs to 90.2%, which sounds great until you see that recall falls from 65% to 4%. It finds just 12 of the 305 genuinely high cost rows. That is not a useful trade.

Table 3

MLP Ablations (Test Months, Change Against the Full Model)

Version	Params	Recall	Change	ROC-AUC	Change	Accuracy
Full MLP (128-64, dropout, class weights)	10,690	.653		.755		.710

Version	Params	Recall	Change	ROC-AUC	Change	Accuracy
No dropout	10,690	.633	-.020	.754	-.001	.720
Half the hidden units (64-32)	3,298	.639	-.013	.755	.000	.730
No class weights	10,690	.039	-.613	.746	-.009	.902

Note. Recall is measured for the high cost class. All versions retained their epoch 1 weights.

We grouped all 869 test mistakes from the MLP into four categories. We used prior year spending compared with the training median and 90th percentile, then looked more closely at eight cases, two from each group. They're recorded in outputs/metrics.json. The biggest category, 589 cases, was overlap: the member looked risky but ended up low cost. Another 174 were false alarms after a costly previous year that didn't repeat. The misses were split. Fifty five had warning signs but fell just below the cutoff, with a median score of 0.42. One 48 year old with three conditions got 0.481 and later cost $6,628. That's partly a threshold issue. The other 51 were harder to see coming. One 29 year old had a single condition and $2,197 in prior spending, scored only 0.117, and then cost $6,125. The model doesn't have much to go on for cases like that. It needs better signals, not just more layers.

The group results also bothered me. The model catches 91.4% of high cost members who are 65 or older, but only 33.7% of those ages 45 to 64. For ages 18 to 44 it catches none. That younger group has 734 test rows, 4.5% are high cost and just 1.1% get flagged at all. Medicaid recall is 25%, while Medicare Advantage is 91.8%. The two counties are much closer, 63.1% and 66.5%. Some of the difference reflects how we built the data. Seniors really are high cost more often here, 19.4% compared with 4.5% for younger members. Still, zero recall for young adults means the planner would miss all of their outreach opportunities.

Ethical and Responsible AI Reflection

Overconfidence and fairness. The group results show outreach could end up focused mostly on older members and people with Medicare Advantage. On top of that, the weighted

models overstate risk by roughly two or three times. A care manager hearing a 60% prediction might assume the member really has that chance of an expensive quarter, when the observed rate is closer to 15%. That's a serious communication problem.

Dataset artifacts. We made the data generator ourselves. So these models are learning the patterns we programmed into a fake population, not discovering some universal healthcare truth. The synthetic scans are more consistent than actual paperwork too. And the target has its own issue: cost isn't the same thing as medical need. Obermeyer et al. (2019) found that using healthcare spending as a proxy for need caused an algorithm to underestimate the needs of Black patients, because less had historically been spent on their care. My target could run into the same kind of problem.

Mitigations. We would use these flags only to help with outreach and provider placement, never to deny care or change prices. I'd also send scans below 70% confidence to a person. In AI Part 2, that rule auto routed 99.3% of messy scans without a wrong route. For member predictions, a large MC Dropout spread would trigger human review too. The age gap needs its own check, maybe different cutoffs by age group, more useful signals than age alone, and a review of young members whose ER visits suddenly rise. And before anyone starts quoting risk percentages, we would calibrate the scores and keep tracking recall by group each month, especially after the data shifts.

Conclusion

The neural network ended up tied with the simpler Part 2 model for predicting expensive quarters. So logistic regression keeps its spot as champion. The CNN helped when we had only clean scans to train on, but data augmentation mattered a lot. For claims, class weighting made a bigger difference than the network architecture itself. And the calibration results were a good

reminder: these scores are useful for ranking who needs a closer look, not for telling somebody their exact odds.

References

Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning* (Vol. 48, pp. 1050–1059). PMLR.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems* (Vol. 35, pp. 507–520). Curran Associates.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436–444. https://doi.org/10.1038/nature14539

Ng, A. Y., & Jordan, M. I. (2002). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems* (Vol. 14, pp. 841–848). MIT Press.

Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science, 366*(6464), 447–453. https://doi.org/10.1126/science.aax2342

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE, 77*(2), 257–286. https://doi.org/10.1109/5.18626

Simonyan, K., Vedaldi, A., & Zisserman, A. (2014). *Deep inside convolutional networks: Visualising image classification models and saliency maps* (arXiv:1312.6034). arXiv. https://doi.org/10.48550/arXiv.1312.6034